

Projektowanie Oprogramowania Systemów - 3			
Projekt	Automatyczny pomiar funkcji		
Wykonujący:	Piotr Krzyżanowski 193675 Hubert Kowalski 193638	Ocena:	
Specjalność:	Inżynieria dźwięku i obrazu		
Data wykonania ćwiczenia:	17.06.2026		

1. Opis wykonanych operacji

W ramach realizacji punktu trzeciego, opracowano krótką specyfikację techniczną. Została ona umieszczona w repozytorium GitHub w pliku *specyfikacja_techiczna.md* w folderze */docs*. Dokument ten podsumowuje zaprojektowany model UML, na który składają się m.in. diagram komponentów oraz diagram klas.

Dodatkowo przeprowadzono analizę używanych bibliotek i narzędzi do realizacji zadania. Wskazano wykorzystanie środowisk Python i C++, bibliotek matematycznych i sygnałowych (*numpy*, *scipy.signal*) oraz wybrano narzędzia pozwalające zrealizować kolejne etapy projektu: framework *pytest* do testów jednostkowych, narzędzie *cProfile* do optymalizacji oraz paczkę *pdoc/Sphinx* do generowania dokumentacji HTML. Zdefiniowano również harmonogram i podział ról w zespole, w ramach którego Piotr pełni rolę Maintainera, a Hubert odpowiada za refaktoryzację, dokumentację HTML, testy i profilowanie wydajnościowe.

2. Dowody realizacji (zrzuty ekranu i diagramy):

Diagram komponentów:

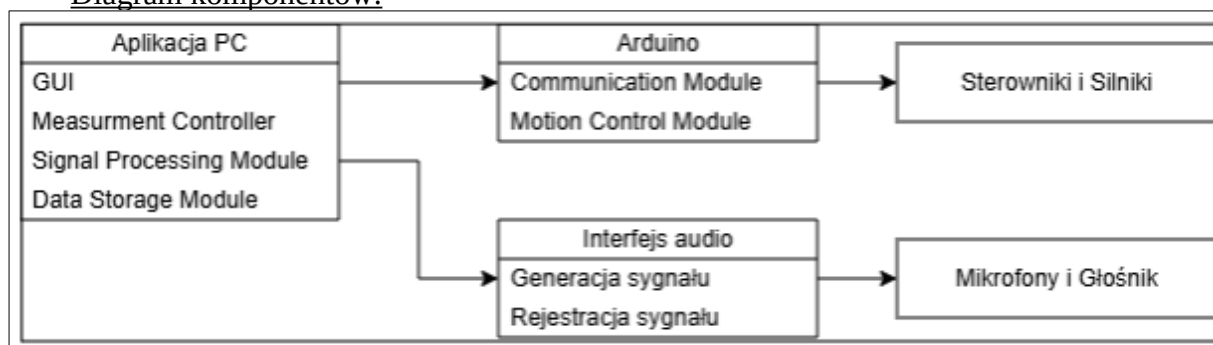
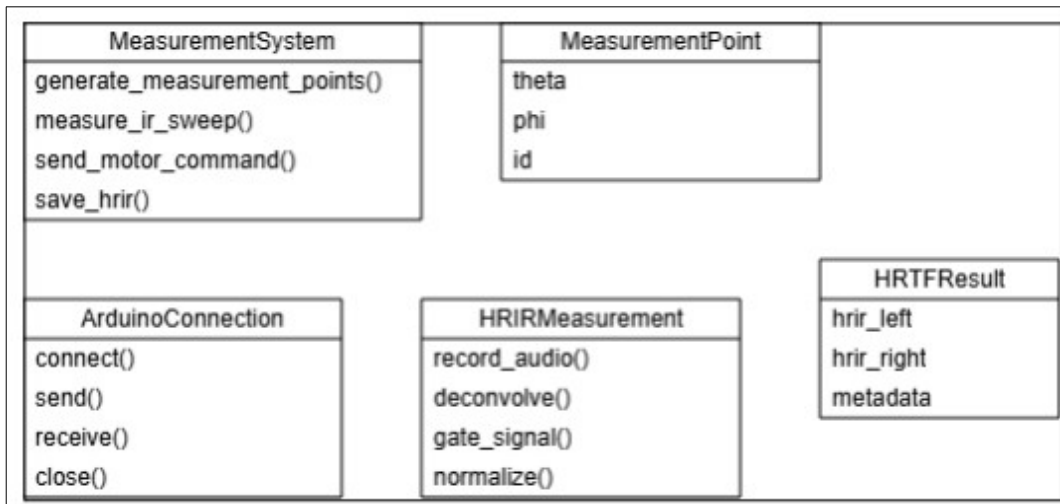


Diagram klas:



Fragment specyfikacji technicznej:

Files | ProjektPOS / docs / specyfikacja_tekniczna.md

whoTisPK Drobne zmiany w plikach zmiany w plikach (zedytuj | loguj)

Preview | Code | Blame | 38 Lines (28 loc) | 3.42 KB

Specyfikacja Techniczna Projektu: Automatyczny pomiar funkcji HRTF

1. Model UML projektowanego rozwiązania

(W tym miejscu diagram komponentów oraz diagram klas)

Architektura systemu opiera się na rozdzieleniu warstwy sterującej (komputer PC) oraz wykonawczej (Arduino). Zgodnie z opracowanym diagramem klas, kluczowe elementy systemu to:

- Modelowanie sprzętu: Klasa `ArduinoConnection` odpowiadająca za protokół UART oraz klasy obsługujące interfejs audio.
- Logika pomiarowa: `HRTFMeasurement` obsługująca akwizycję i dekonwolucję sygnału, a także algorytm generowania siatki sferycznej. Rozwiązanie zapewnia modularność – logika sterowania silnikami jest odseparowana od toru przetwarzania sygnałów audio.

2. Analiza bibliotek i narzędzi

Projekt wykorzystuje następujący stos technologiczny:

Narzędzia i biblioteki bazowe (już zaimplementowane):

- Język programowania: **Python** (aplikacja PC), **C++** (mikrokontroler Arduino).
- Numpy**: Wykorzystywana do zaawansowanych obliczeń matematycznych (np. generowanie punktów na sferze metodą Deserno).
- SciPy**: Niezbędna do cyfrowego przetwarzania sygnałów (operacje splotu/dekonwolucji oraz aplikacja okien wygładzających Tukeya).
- Sounddevice & Soundfile**: Używane do jednoczesnego odczytania sygnału typu sweep-sine i rejestracji odpowiedzi z mikrofonów.
- Pyserial**: Umożliwia synchronizowaną komunikację szeregową PC <-> Arduino.
- Matplotlib**: Służy do wizualizacji siatki wygenerowanych punktów pomiarowych.

Dodatkowe narzędzia do realizacji założeń POS:

- pellet / Sphinx**: Biblioteki posłużą do automatycznego generowania dokumentacji HTML wprost z komentarzy (docstringów) dodanych do klas i metod.
- pytest**: Framework wybrany do realizacji i automatyzacji testów jednostkowych (np. walidacji funkcji wyliczających liczbę kroków silnika).
- cProfile**: Wbudowany w Pythona profiler posłuży do poszukiwania "hot spotów" wydajnościowych (m.in. w algorytmach splotu sygnałów audio).

3. Harmonogram i podział pracy w zespole